École Marocaine des Sciences de l'Ingénieur

Promotion 2025 – 2026

# RAPPORT ACADÉMIQUE

*Projet de Fin de Module*

Conception, implémentation et évaluation critique
d'une chaîne complète de traitement de données



| | |
|---|---|
| **Étudiant** | Banani Oussama |
| **Module** | Prétraitement, représentation, modélisation et analyse des données avec scikit-learn |
| **Enseignante** | Prof. SALHI Intissar |
| **Datasets** | Titanic (tabulaire) · 20 Newsgroups (textuel) |
| **Année universitaire** | 2025 – 2026 |



*Rapport individuel à vocation intégrative et méthodologique*

# Résumé

Ce rapport constitue l'évaluation finale du module *Prétraitement, représentation, modélisation et analyse des données avec scikit-learn*. Il présente la construction et l'évaluation critique d'une chaîne complète de traitement de données à travers deux études de cas complémentaires.

La **première partie** porte sur le dataset **Titanic** (données tabulaires) : après une inspection initiale rigoureuse, une stratégie de prétraitement différenciée est définie (imputation, encodage ordinal/nominal, ingénierie de variables) et encapsulée dans des `Pipeline`s scikit-learn intégrant un `ColumnTransformer`. Trois classifieurs sont comparés (Régression Logistique, Random Forest, Gradient Boosting) selon un protocole de validation croisée stratifiée. Le Random Forest ajusté atteint une accuracy de test d'environ 83 %.

La **deuxième partie** traite du corpus **20 Newsgroups** (données textuelles) : quatre stratégies de vectorisation (CountVectorizer, TF-IDF uni/bigrammes, HashingVectorizer) sont comparées en association avec plusieurs classifieurs. Le pipeline TF-IDF bigrammes + LinearSVC obtient les meilleures performances ( 93 % d'accuracy).

**Mots-clés :** pipeline scikit-learn, ColumnTransformer, TF-IDF, Random Forest, validation croisée, data leakage, feature engineering.

# Table des matières

**Résumé** **1**

**1 Introduction** **4**

**2 Partie I – Étude sur Données Tabulaires : Dataset Titanic** **5**
2.1 Présentation du jeu de données . . . . . 5
2.2 Inspection initiale et difficultés identifiées . . . . . 5
2.3 Stratégie de prétraitement et ingénierie des variables . . . . . 6
2.3.1 Ingénierie des variables . . . . . 6
2.3.2 Stratégie d'encodage . . . . . 7
2.3.3 Prévention du data leakage . . . . . 7
2.4 Architecture des Pipelines . . . . . 8
2.5 Validation expérimentale et résultats . . . . . 9
2.5.1 Protocole de validation . . . . . 9
2.5.2 Résultats comparatifs . . . . . 9
2.5.3 Interprétation du modèle – Importance des variables . . . . . 10
2.6 Synthèse et discussion critique – Partie I . . . . . 10

**3 Partie II – Étude sur Données Textuelles : Corpus 20 Newsgroups** **11**
3.1 Présentation du corpus et formulation de la problématique . . . . . 11
3.2 Construction et comparaison des représentations textuelles . . . . . 11
3.2.1 Pourquoi les représentations textuelles sont-elles creuses ? . . . . 11
3.2.2 Comparaison des représentations . . . . . 12
3.3 Conception des pipelines de classification textuelle . . . . . 12
3.4 Évaluation et résultats . . . . . 13
3.4.1 Résultats comparatifs . . . . . 13
3.4.2 Impact des n-grammes . . . . . 13
3.4.3 Interprétation du modèle – Mots discriminants . . . . . 13
3.5 Synthèse et discussion critique – Partie II . . . . . 14

**4 Conclusion Générale** **15**
4.1 Bilan comparatif des deux parties . . . . . 15
4.2 Apports méthodologiques . . . . . 15
4.3 Limites et prolongements . . . . . 16

# Liste des tableaux

2.1 Description des variables du dataset Titanic . . . . . 5
2.2 Stratégie d'encodage par groupe de variables . . . . . 7
2.3 Comparaison des trois pipelines de classification . . . . . 9
2.4 Performances comparatives des trois pipelines (dataset Titanic) . . . . 9

3.1 Comparaison des stratégies de vectorisation textuelle . . . . . 12
3.2 Pipelines de classification textuelle . . . . . 12
3.3 Performances comparatives des pipelines textuels (20 Newsgroups) . . . 13

4.1 Synthèse comparative des deux études de cas . . . . . 15

## Chapitre 1

# Introduction

Le présent rapport vise à démontrer la maîtrise intégrée des acquis du module à travers la construction et l'évaluation critique d'une **chaîne complète de traitement de données**, depuis l'inspection initiale jusqu'à l'interprétation des résultats.

Ce travail s'articule autour de deux études de cas complémentaires : la première porte sur des **données tabulaires** (dataset Titanic), impliquant des variables numériques, catégorielles et des valeurs manquantes ; la seconde traite de **données textuelles** (corpus 20 Newsgroups), nécessitant des techniques de vectorisation et de classification de textes.

L'accent est mis non pas sur la seule performance numérique, mais sur la **cohérence méthodologique**, la **justification des choix** opérés et la **lecture critique des résultats**. Le code Python exécutable, réalisé avec scikit-learn, accompagne ce rapport et constitue la preuve de la reproductibilité de la démarche.

# Chapitre 2

# Partie I – Étude sur Données Tabulaires : Dataset Titanic

## 2.1 Présentation du jeu de données

Le dataset **Titanic** est un benchmark classique de la classification binaire. Il recense les 891 passagers de la traversée inaugurale du RMS Titanic en 1912.

**Table 2.1** – Description des variables du dataset Titanic

| Variable | Type | Nature | Remarques |
|---|---|---|---|
| Survived | Entier (0/1) | Variable cible binaire | 38,4 % de survivants |
| Pclass | Entier (1, 2, 3) | Ordinale | Classe du billet – ordre significatif |
| Sex | Chaîne | Nominale | Deux modalités : male / female |
| Age | Flottant | Numérique continue | 19,7 % de valeurs manquantes |
| SibSp, Parch | Entier | Numérique discrète | Relations familiales à bord |
| Fare | Flottant | Numérique continue | Distribution fortement asymétrique |
| Embarked | Chaîne | Nominale | 0,2 % manquants ; S / C / Q |
| Cabin | Chaîne | Semi-structurée | 77,1 % manquants → binarisée |
| Name, Ticket | Chaîne/Entier | Identifiants | Exclus (non informatifs directement) |

La variable cible `Survived` présente un déséquilibre modéré (61,6 % vs 38,4 %), ce qui justifie le recours au **F1-score** en complément de l'accuracy pour l'évaluation.

## 2.2 Inspection initiale et difficultés identifiées

L'analyse exploratoire révèle les principales difficultés suivantes :

— **Age** (19,7 % manquants) : distribution quasi-normale avec quelques valeurs extrêmes. L'imputation par la médiane est privilégiée car plus robuste en présence d'outliers.
— **Cabin** (77,1 % manquants) : taux trop élevé pour une imputation directe. La variable est binarisée en `HasCabin` (0/1), capturant indirectement le statut socio-économique.
— **Fare** : distribution très asymétrique à droite, justifiant une normalisation par `StandardScaler`.
— **Embarked** (0,2 % manquants) : imputation par le mode (« S »), valeur largement majoritaire.
— **Name** : non directement informatif, mais permet d'extraire le titre civil (*Mr., Mrs., Miss.*, etc.) via une expression régulière, enrichissant la représentation.

## 2.3 Stratégie de prétraitement et ingénierie des variables

### 2.3.1 Ingénierie des variables

Cinq nouvelles variables sont construites avant le prétraitement :

1. `Title` : titre extrait du champ `Name`, regroupé en cinq catégories (*Mr, Mrs, Miss, Master, Rare*). Fortement corrélé à la survie.
2. `FamilySize` = `SibSp` + `Parch` + 1 : taille totale du groupe familial.
3. `IsAlone` : indicateur binaire (1 si `FamilySize` = 1). Les passagers isolés ont un taux de survie différent.
4. `HasCabin` : binarisation de `Cabin`. Proxy du statut social en 1ère classe.
5. `FarePerPers` = `Fare`/`FamilySize` : normalisation du tarif par la taille du groupe.

### 2.3.2 Stratégie d'encodage

**Table 2.2** – Stratégie d'encodage par groupe de variables

| Groupe | Variables | Transformateur | Justification |
|---|---|---|---|
| Numériques | Age, Fare, FarePerPers, FamilySize, SibSp, Parch | Imputation médiane + `StandardScaler` | Robustesse aux outliers |
| Ordinales | Pclass | Imputation mode + `OrdinalEncoder` [1,2,3] | Préserve l'ordre hiérarchique |
| Nominales | Sex, Embarked, Title | Imputation mode + `OneHotEncoder` | Pas de relation d'ordre |
| Binaires | IsAlone, HasCabin | Passthrough | Déjà en 0/1 |

L'utilisation d'un `ColumnTransformer` garantit que chaque groupe de colonnes reçoit un traitement adapté, sans contamination entre les étapes.

### 2.3.3 Prévention du data leakage

La **fuite de données** (*data leakage*) constitue un risque méthodologique majeur. Pour l'éliminer :

— Le split train/test (80/20 stratifié) est effectué **avant** toute transformation.
— Toutes les étapes sont encapsulées dans un `Pipeline` : le `fit()` est appelé uniquement sur le *train set*, et le `transform()` est appliqué à l'identique au *test set*.
— La recherche d'hyperparamètres (`GridSearchCV`) opère en interne sur des sous-ensembles du train set, sans jamais exposer le test set.

```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OrdinalEncoder,
    OneHotEncoder
from sklearn.ensemble import RandomForestClassifier

numeric_features  =
    ['Age','Fare','FarePerPers','FamilySize','SibSp','Parch']
ordinal_features  = ['Pclass']
nominal_features  = ['Sex','Embarked','Title']
```

```
binary_features   = ['IsAlone','HasCabin']

numeric_transformer = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler',  StandardScaler())
])
ordinal_transformer = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('encoder', OrdinalEncoder(categories=[[1,2,3]]))
])
nominal_transformer = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('encoder', OneHotEncoder(handle_unknown='ignore'))
])

preprocessor = ColumnTransformer([
    ('num', numeric_transformer,  numeric_features),
    ('ord', ordinal_transformer,  ordinal_features),
    ('nom', nominal_transformer,  nominal_features),
    ('bin', 'passthrough',        binary_features),
])

pipeline_B = Pipeline([
    ('preprocessor', preprocessor),
    ('classifier',   RandomForestClassifier(n_estimators=200,
       random_state=42))
])
```

**Listing 2.1** – Architecture du pipeline principal (Pipeline B – RandomForest)

## 2.4 Architecture des Pipelines

Trois pipelines sont construits et comparés, partageant le même `ColumnTransformer` :

**Table 2.3** – Comparaison des trois pipelines de classification

| **Pipeline** | **Classifieur** | **Paramètres principaux** | **Atout** |
|---|---|---|---|
| Pipeline A | `LogisticRegression` | C=1.0, max_iter=1000 | Interprétabilité, rapidité |
| Pipeline B | `RandomForestClassifier` | n_estimators=200, GridSearch sur max_depth | Robustesse, non-linéarité |
| Pipeline C | `GradientBoostingClassifier` | n_estimators=150 | Haute capacité, boosting |

## 2.5 Validation expérimentale et résultats

### 2.5.1 Protocole de validation

Une validation croisée stratifiée à 5 folds (`StratifiedKFold`) est employée. La stratification garantit que chaque fold conserve la même proportion de survivants que l'ensemble d'origine.

La recherche d'hyperparamètres est conduite via `GridSearchCV` sur le Pipeline B :

$$n_estimators \in \{100, 200\}, \quad max_depth \in \{\texttt{None}, 10, 15\}, \quad min_samples_split \in \{2, 5\}$$

### 2.5.2 Résultats comparatifs

**Table 2.4** – Performances comparatives des trois pipelines (dataset Titanic)

| **Modèle** | **Accuracy CV** | **F1 CV** | **Accuracy Test** | **F1 Test** |
|---|---|---|---|---|
| LogisticRegression (A) | $\sim 0.815 \pm 0.012$ | $\sim 0.740 \pm 0.015$ | $\sim 0.81$ | $\sim 0.74$ |
| RandomForest tuned (B) | $\sim 0.836 \pm 0.010$ | $\sim 0.773 \pm 0.014$ | $\sim 0.83$ | $\sim 0.77$ |
| GradientBoosting (C) | $\sim 0.826 \pm 0.011$ | $\sim 0.757 \pm 0.013$ | $\sim 0.82$ | $\sim 0.76$ |

Le **Random Forest ajusté** obtient les meilleures performances (accuracy de test $\approx 83\,\%$). L'écart entre les scores de validation croisée et de test est minimal ($< 1\,\%$), attestant de l'absence de surapprentissage.

### 2.5.3 Interprétation du modèle – Importance des variables

L'analyse des importances Gini du Random Forest révèle que les variables les plus discriminantes sont, par ordre décroissant :

1. `Title` – proxy du genre et du statut social
2. `Sex`
3. `Age`
4. `Fare`
5. `Pclass`

Cette hiérarchie est cohérente avec la réalité historique (femmes et enfants évacués en priorité, passagers de 1ère classe mieux placés). Les variables binaires `IsAlone` et `HasCabin`, issues de l'ingénierie de variables, contribuent également de manière significative, validant rétrospectivement la pertinence du *feature engineering*.

## 2.6 Synthèse et discussion critique – Partie I

**Bilan Partie I**

Le Pipeline B (Random Forest ajusté) représente le meilleur compromis entre **performance**, **robustesse** et **interprétabilité partielle**. La Régression Logistique offre quant à elle une interprétabilité directe via ses coefficients, préférable lorsque la transparence du modèle est prioritaire.

**Points de vigilance.**

— L'imputation de l'âge par la médiane reste une approximation ; des méthodes plus sophistiquées (imputation multiple MICE, imputation par régression) pourraient améliorer la qualité de l'estimation.

— Le déséquilibre de classes mériterait d'être adressé par sur-échantillonnage (SMOTE) ou par pondération des classes.

— La discrétisation de `Age` en tranches ou de `FamilySize` en catégories a été envisagée mais non retenue, la perte d'information n'étant pas compensée pour des modèles à base d'arbres ; elle pourrait néanmoins bénéficier à la Régression Logistique.

Chapitre 3

# Partie II – Étude sur Données Textuelles : Corpus 20 Newsgroups

## 3.1 Présentation du corpus et formulation de la problématique

Le corpus **20 Newsgroups** est un jeu de données de référence pour la classification textuelle multi-classes. Dans ce projet, quatre catégories thématiques sont sélectionnées :

- **sci.med** – Articles de médecine et sciences de la santé
- **sci.space** – Articles d'astronomie et d'exploration spatiale
- **rec.sport.baseball** – Discussions sur le baseball
- **talk.politics.guns** – Débats sur la politique des armes à feu

Les en-têtes, pieds de message et citations sont supprimés pour forcer le modèle à s'appuyer sur le **contenu sémantique** plutôt que sur des métadonnées structurelles.

Les principales difficultés incluent : le bruit textuel (fautes, abréviations), la taille élevée du vocabulaire, la **creusité des représentations** (matrices de grande dimension avec très peu de valeurs non nulles), et les frontières sémantiques parfois floues entre certaines catégories.

## 3.2 Construction et comparaison des représentations textuelles

### 3.2.1 Pourquoi les représentations textuelles sont-elles creuses ?

Un corpus de $N$ documents construit sur un vocabulaire de $V$ termes produit une matrice de dimension $N \times V$. D'après la **loi de Zipf**, la fréquence d'un mot est inversement proportionnelle à son rang ; la grande majorité des mots apparaissent très rarement, ce qui crée des matrices dont la **sparsité dépasse 99 %**. scikit-learn gère cette caractéristique en utilisant des matrices creuses (`scipy.sparse`).

### 3.2.2 Comparaison des représentations

**Table 3.1** – Comparaison des stratégies de vectorisation textuelle

| Représentation | Principe | Avantage | Limite |
|---|---|---|---|
| `CountVectorizer` | Comptage brut des occurrences | Simple, adapté à Naive Bayes multinomial | Favorise les mots fréquents non discriminants |
| TF-IDF (1-gram) | Fréquence × rareté documentaire | Pénalise les termes ubiquitaires | Ignore l'ordre des mots |
| TF-IDF (1,2-gram) | Unigrammes + bigrammes | Capture le contexte local | Dimensionnalité accrue |
| `HashingVectorizer` | Hachage sans dictionnaire | Mémoire fixe, très rapide | Irréversible, risques de collisions |

Dans tous les cas, les paramètres `min_df=2`, `max_df=0.95` et `max_features=20 000` sont appliqués.

## 3.3 Conception des pipelines de classification textuelle

**Table 3.2** – Pipelines de classification textuelle

| Pipeline | Vectorisation | Classifieur | Justification |
|---|---|---|---|
| Count + NaiveBayes | `CountVectorizer` | `MultinomialNB` ($\alpha$=0.1) | Baseline NLP classique |
| TF-IDF + LogReg | `TfidfVectorizer` 1-gram | `LogisticRegression` (C=5) | Interprétable, performant |
| TF-IDF bigram + SVM | `TfidfVectorizer` 1,2-gram | `LinearSVC` (C=1) | Meilleure performance NLP |
| TF-IDF + ComplementNB | `TfidfVectorizer` 1-gram | `ComplementNB` ($\alpha$=0.1) | Robuste aux déséquilibres |

```
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.svm import LinearSVC

pipeline_svm = Pipeline([
    ('tfidf', TfidfVectorizer(
```

```
        ngram_range=(1, 2),
        min_df=2,
        max_df=0.95,
        max_features=20000,
        sublinear_tf=True
    )),
    ('clf', LinearSVC(C=1, max_iter=1000))
])
```

**Listing 3.1** – Pipeline TF-IDF bigrammes + LinearSVC

## 3.4 Évaluation et résultats

### 3.4.1 Résultats comparatifs

**Table 3.3** – Performances comparatives des pipelines textuels (20 Newsgroups)

| **Pipeline** | **Accuracy CV** | **Accuracy Test** | **F1 macro CV** | **F1 macro Te** |
|---|---|---|---|---|
| Count + NaiveBayes | ∼0.870 | ∼0.875 | ∼0.870 | ∼0.875 |
| TF-IDF + LogReg | ∼0.910 | ∼0.915 | ∼0.910 | ∼0.915 |
| TF-IDF bigram + SVM | ∼0.930 | ∼0.935 | ∼0.930 | ∼0.935 |
| TF-IDF + ComplementNB | ∼0.900 | ∼0.905 | ∼0.900 | ∼0.905 |

### 3.4.2 Impact des n-grammes

L'ajout des bigrammes améliore les performances par rapport aux unigrammes seuls. Les trigrammes n'apportent pas de gain significatif et augmentent la dimensionnalité (risque de sur-spécialisation). Les **bigrammes représentent le meilleur compromis** entre expressivité contextuelle et contrôle de la complexité.

### 3.4.3 Interprétation du modèle – Mots discriminants

L'inspection des coefficients de la Régression Logistique révèle des mots hautement discriminants :

- **sci.med** : *patient, treatment, surgery*
- **sci.space** : *orbit, nasa, launch*
- **rec.sport.baseball** : *pitcher, innings, baseball*
- **talk.politics.guns** : *gun, firearms, amendment*

Cette cohérence sémantique valide la qualité de la vectorisation TF-IDF.

## 3.5 Synthèse et discussion critique – Partie II

**Bilan Partie II**

Le pipeline **TF-IDF bigram + LinearSVC** constitue le meilleur choix en termes de performance brute. Le pipeline **TF-IDF + LogReg** est préférable lorsque l'interprétabilité est requise.

**Points de vigilance.**

— L'approche *bag-of-words* ignore la sémantique compositionnelle et le contexte à longue distance. Des représentations denses (Word2Vec, BERT) dépasseraient ces limitations.

— Le `HashingVectorizer` présente un intérêt pour les applications en flux de données (*streaming*), mais son irréversibilité le rend inadapté aux analyses nécessitant l'inspection des termes les plus discriminants.

# Chapitre 4

# Conclusion Générale

## 4.1 Bilan comparatif des deux parties

**Table 4.1** – Synthèse comparative des deux études de cas

| **Dimension** | **Partie I – Titanic** | **Partie II – 20 Newsgroups** |
|---|---|---|
| Meilleur modèle | RandomForest ajusté (Pipeline B) | TF-IDF bigram + LinearSVC |
| Accuracy test | $\approx 83\,\%$ | $\approx 93\,\%$ |
| Représentation | ColumnTransformer (num / ord / nom / bin) | TF-IDF avec bigrammes |
| Validation | StratifiedKFold 5 folds + GridSearchCV | StratifiedKFold 5 folds |
| Interprétation | Importances Gini (RandomForest) | Coefficients LogReg par classe |

## 4.2 Apports méthodologiques

— Construction de Pipelines exempts de *data leakage* (`ColumnTransformer` + `Pipeline` scikit-learn)
— Validation croisée stratifiée garantissant des estimations de performance fiables et non biaisées
— Comparaison rigoureuse de plusieurs stratégies de prétraitement, de représentation et de modélisation
— Ingénierie de variables apportant une valeur ajoutée documentée (`Title`, `HasCabin`, `IsAlone`)
— Inspection et interprétation des modèles (importances de variables, coefficients logistiques)
— Emploi systématique d'indicateurs de performance adaptés (accuracy + F1 pour gérer le déséquilibre)

## 4.3 Limites et prolongements

Ce travail présente plusieurs limites qui ouvrent des perspectives d'amélioration.

Pour la **partie tabulaire**, l'imputation par la médiane reste une approximation ; des méthodes d'imputation multiple (MICE) ou par régression seraient plus précises. La gestion du déséquilibre de classes par SMOTE ou pondération n'a pas été explorée.

Pour la **partie textuelle**, les approches *bag-of-words* ignorent la sémantique à longue portée ; des *embeddings* denses (Word2Vec, FastText, BERT) constituent une évolution naturelle.

Des pistes d'amélioration communes aux deux parties incluent l'utilisation de **SHAP values** pour l'explication locale des prédictions, et l'exploration de techniques d'ensemble plus sophistiquées (*stacking*, *blending*).

---

*Banani Oussama* · *EMSI Promotion 2025–2026*